

Sesión 8: Recorridos y Eliminación en Árboles Binarios

Estrategias de Exploración Recursiva (Inorden, Preorden, Postorden) y Algoritmos Complejos para Reestructuración de Memoria Dinámica en un ABB

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

| Agenda del Día (1/2)

-  Teoría General de Recorridos en Estructuras No Lineales
-  Recorrido Inorden: Extracción de Datos de Forma Ordenada
-  Recorridos Preorden y Postorden: Casos de Aplicación
-  Introducción a la Eliminación de Nodos en un ABB



Complejidad Operativa

Analizaremos paso a paso cómo desconectar nodos de la memoria manteniendo intactas las propiedades jerárquicas esenciales.

Agenda del Día (2/2)



Los Tres Casos

Análisis algorítmico exhaustivo para nodos hoja, nodos con un hijo y nodos con dos hijos directos.



Código en C#

Implementación pura orientada a objetos en .NET para la reestructuración dinámica del árbol.



Organización

Valor de la unidad: Estructuración limpia y legible para asegurar el mantenimiento del software.



Recorridos Recursivos en Árboles

Estrategias científicas para visitar cada elemento jerárquico exactamente una vez

| ¿Qué significa recorrer un Árbol?

A diferencia de las listas secuenciales donde solo hay un camino (de inicio a fin), en las estructuras no lineales existen ****múltiples formas de visitar los elementos****.

Un recorrido es un proceso sistemático que visita todos los nodos del árbol de manera ordenada, transformando la estructura jerárquica bidimensional en una salida lineal lógica.

1. Recorrido Inorden (In-Order)

Sigue estrictamente la siguiente secuencia de ejecución recursiva:

1. Procesar recursivamente el Subárbol Izquierdo.
2. Visitar/Imprimir el Nodo Actual (Raíz local).
3. Procesar recursivamente el Subárbol Derecho.

i Propiedad Clave en un ABB:

El recorrido Inorden de un Árbol Binario de Búsqueda devolverá ****siempre**** los elementos ordenados de menor a mayor de forma matemática.

Código: Implementación del Recorrido Inorden

```
public void RecorrerInorden() {  
    InordenRecursivo(raiz);  
    Console.WriteLine();  
}  
  
private void InordenRecursivo(NodoArbol actual) {  
    if (actual != null) {  
        InordenRecursivo(actual.Izquierdo); // 1. Izquierda  
        Console.Write(actual.Dato + " "); // 2. Raíz  
        InordenRecursivo(actual.Derecho); // 3. Derecha  
    }  
}
```


2. Recorrido Preorden (Pre-Order)

Sigue una estrategia de procesamiento descendente prioritario:

1. Visitar/Imprimir el Nodo Actual (Raíz local).
2. Procesar recursivamente el Subárbol Izquierdo.
3. Procesar recursivamente el Subárbol Derecho.

****Caso de uso principal:**** Es ideal para realizar copias exactas o clonaciones completas de la estructura del árbol en otro espacio de la memoria heap.

Código: Implementación del Recorrido Preorden

```
public void RecorrerPreorden() {  
    PreordenRecursivo(raiz);  
    Console.WriteLine();  
}  
  
private void PreordenRecursivo(NodoArbol actual) {  
    if (actual != null) {  
        Console.Write(actual.Dato + " "); // 1. Raíz  
        PreordenRecursivo(actual.Izquierdo); // 2. Izquierda  
        PreordenRecursivo(actual.Derecho); // 3. Derecha  
    }  
}
```


3. Recorrido Postorden (Post-Order)

Sigue una estrategia de procesamiento ascendente desde las hojas hacia el origen:

1. Procesar recursivamente el Subárbol Izquierdo.
2. Procesar recursivamente el Subárbol Derecho.
3. Visitar/Imprimir el Nodo Actual (Raíz local).

****Caso de uso principal:**** Liberación sistemática de memoria, eliminación física de estructuras completas en lenguajes sin recolector de basura automático, o evaluación de expresiones algebraicas complejas.

Código: Implementación del Recorrido Postorden

```
public void RecorrerPostorden() {  
    PostordenRecursivo(raiz);  
    Console.WriteLine();  
}  
  
private void PostordenRecursivo(NodoArbol actual) {  
    if (actual != null) {  
        PostordenRecursivo(actual.Izquierdo); // 1. Izquierda  
        PostordenRecursivo(actual.Derecho);   // 2. Derecha  
        Console.Write(actual.Dato + " ");    // 3. Raíz  
    }  
}
```




Algoritmia de Eliminación en un ABB

El desafío de borrar un nodo sin destruir la jerarquía y el ordenamiento del sistema

El Desafío de la Desconexión Dinámica

Insertar y buscar elementos es una operación relativamente directa. Sin embargo, eliminar un nodo requiere ****reestructurar los punteros**** para evitar dejar subárboles huérfanos.

La estrategia se divide en ****tres escenarios operativos excluyentes**** según el grado de ramificación del elemento a borrar:

Caso 1

El nodo es una Hoja Terminal (0 hijos)

Caso 2

El nodo posee un Único Hijo (1 hijo)

Caso 3

El nodo posee Estructura Completa (2 hijos)

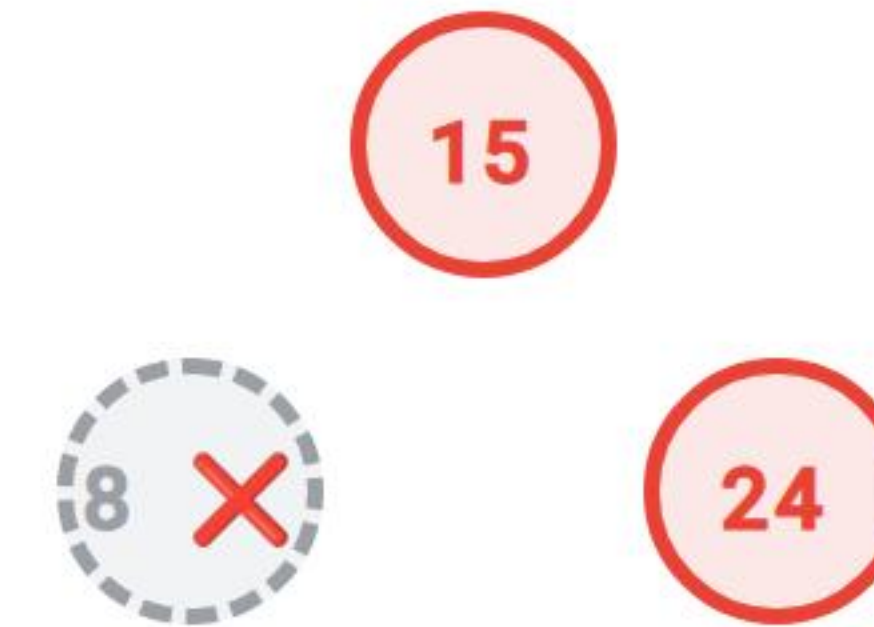
Caso 1: El Nodo es una Hoja (0 hijos)

Es el escenario operativo más simple de resolver.

Mecánica Algorítmica:

Como el elemento no tiene descendencia, simplemente cambiamos el puntero de su nodo padre que apuntaba hacia él, asignándole el valor `**`null`**`.

El objeto queda aislado y el recolector de basura de .NET libera de forma automática su memoria.



Caso 2: El Nodo tiene un Único Hijo (1 hijo)

El elemento a eliminar actúa como puente intermedio en la jerarquía vertical.

****Mecánica de Bypass:**** El abuelo debe adoptar de forma directa al nieto. Enlazamos el puntero del nodo padre del elemento que vamos a eliminar para que apunte directamente al único hijo de este.

✓ No importa si el hijo es el izquierdo o el derecho, el bypass mantiene intacto el orden del ABB.

Caso 3: El Nodo tiene Dos Hijos

Es el escenario más complejo de la sesión. No podemos hacer un bypass simple porque el padre del elemento que eliminamos no puede adoptar a dos hijos en una sola rama.

Estrategia de Reemplazo por Sucesor:

Debemos encontrar un sustituto intermedio. Buscamos ****el valor mínimo del subárbol derecho**** (el sucesor inorden). Copiamos su valor al nodo que deseamos eliminar y luego borramos físicamente ese nodo sucesor del subárbol derecho (el cual siempre caerá en el Caso 1 o Caso 2).



Traducción a Código de Eliminación C#

Estructura completa de la rutina de reestructuración dinámica

Código: Punto de Entrada de Eliminación

Protección y encapsulamiento formal de la interfaz del objeto:

```
public void Eliminar(int valor) {  
    raiz = EliminarRecursivo(raiz, valor);  
}
```

Al igual que en el algoritmo de inserción, el método público se encarga de reasignar el estado de la propiedad privada raiz tras procesar la cascada recursiva.

Código: Buscar el Nodo a Eliminar (1/3)

```
private NodoArbol EliminarRecursoivo(NodoArbol actual, int valor) {  
    if (actual == null) return null; // Elemento no encontrado  
  
    // Navegación lógica para localizar el objetivo  
    if (valor < actual.Dato) {  
        actual.Izquierdo = EliminarRecursoivo(actual.Izquierdo, valor);  
    } else if (valor > actual.Dato) {  
        actual.Derecho = EliminarRecursoivo(actual.Derecho, valor);  
    } else {  
        // ¡Encontrado! Aquí se ejecutan los 3 casos de eliminación  
        // (Continúa en la siguiente diapositiva...)  
    }  
}
```


Código: Lógica de Caso 1 y Caso 2 (2/3)

```
// Caso 1 y Caso 2: Cero o un solo hijo activo
if (actual.Izquierdo == null) {
    return actual.Derecho; // Retorna el hijo derecho (o null si era hoja)
}
if (actual.Derecho == null) {
    return actual.Izquierdo; // Retorna el hijo izquierdo
}

// Caso 3: El nodo tiene dos hijos activos
// (Continúa en la siguiente diapositiva...)
```


Código: Lógica de Caso 3 Complejo (3/3)

```
// Caso 3: Obtener el valor mínimo del subárbol derecho
actual.Dato = ObtenerValorMinimo(actual.Derecho);

// Eliminar físicamente el nodo sucesor duplicado
actual.Derecho = EliminarRecurso(actual.Derecho, actual.Dato);
}
return actual;
}
```


Código: Función de Búsqueda del Valor Mínimo

Para hallar el valor mínimo, nos desplazamos ****siempre hacia el extremo izquierdo**** de ese subárbol:

```
private int ObtenerValorMinimo(NodoArbol nodo) {  
    int minValor = nodo.Dato;  
    while (nodo.Izquierdo != null) {  
        minValor = nodo.Izquierdo.Dato;  
        nodo = nodo.Izquierdo;  
    }  
    return minValor;  
}
```




Simulación e Integración del Sistema

Pruebas unitarias lógicas en consola para validar los estados dinámicos del ABB

Código: Orquestación General del Flujo

```
public static void Main(string[] args) {  
    ArbolBinarioBusqueda arbol = new ArbolBinarioBusqueda();  
    int[] llaves = { 50, 30, 70, 20, 40, 60, 80 };  
    foreach (var k in llaves) arbol.Insertar(k);  
  
    Console.WriteLine("Inorden Inicial: "); arbol.RecorrerInorden(); // 20 30 40 50 60 70 80  
  
    arbol.Eliminar(20); // Borrar Hoja (Caso 1)  
    arbol.Eliminar(30); // Borrar Nodo con un hijo (Caso 2)  
    arbol.Eliminar(50); // Borrar Raíz con dos hijos (Caso 3)  
  
    Console.WriteLine("Inorden Final: "); arbol.RecorrerInorden(); // 40 60 70 80  
}
```


Análisis de Salida y Verificación Estructural

Evaluemos la salida obtenida en la consola de comandos de .NET:

```
Inorden Inicial: 20 30 40 50 60 70 80  
Inorden Final: 40 60 70 80
```

✓ ****Resultado esperado:**** A pesar de haber eliminado tres nodos bajo escenarios de complejidad totalmente distintos, la colección final mantiene un perfecto orden ascendente.



Buenas Prácticas de Ingeniería

Estrategias avanzadas para mitigar fallos por pérdida de referencias dinámicas

Error Crítico: Confundir Sucesor con Predecesor

Un error muy común en las evaluaciones del laboratorio consiste en mezclar los criterios del Caso 3:

Criterio Erróneo: Intentar extraer el valor mínimo del subárbol izquierdo o el valor máximo del subárbol derecho.

⚠️ ****Consecuencia:**** Esto viola de inmediato la propiedad de ordenación del ABB, provocando que las futuras búsquedas de llaves fallen de manera silenciosa al tomar caminos incorrectos.

Valor de la Unidad: Organización



"Cualquier tonto puede escribir código que un ordenador entienda. Los buenos programadores escriben código que los humanos pueden entender."

— Martin Fowler

****Aplicación en Estructuras:**** Implementar rutinas recursivas complejas de forma modular y organizada facilita la depuración de los punteros en sistemas de producción a gran escala.

Conclusiones Generales de la Sesión

Salida Inorden

El recorrido Inorden es la herramienta por excelencia para linealizar y extraer la información de un ABB de forma ordenada y ascendente.

Estrategia del Caso 3

Eliminar un nodo con dos hijos requiere un reemplazo por el valor mínimo de su subárbol derecho para preservar las restricciones estructurales.

Cuidado de Punteros

Capturar adecuadamente los retornos de las funciones recursivas evita dejar nodos huérfanos o perder subárboles enteros en memoria.

Próximo Paso: Sesión 9

Árboles AVL y Rotaciones Simples

En la próxima sesión resolveremos el problema de la degradación de rendimiento lineal mediante el estudio del **Factor de Equilibrio (FE)** y la aplicación de **Rotaciones Simples** (RLL y RRD) en árboles auto-balanceables.

¿Dudas de Recorridos o Eliminación en ABB?

Es momento de actualizar e integrar sus proyectos.

 Foros oficiales en UEDI  Soporte: 3021894750101@ingenieria.usac.edu.gt